

FOTA Implementation Specification — v2

Target MCU: STM32H563 (TrustZone enabled, Cortex-M33)
Delivery mechanism: HTTP POST via `curl` on port 8080
Firmware signing: X.509 certificate (ECDSA-P256) + SHA-256 integrity
Crypto library: wolfSSL v5.7.4 with `WOLFSSL_STATIC_MEMORY` (no heap, Secure world only)
Allocation: Fully static — 8KB static pool in `.bss` for wolfSSL, no `malloc/free`
Status: v2 architecture — Secure-side verification, NS write-only

1. Architecture Overview



Key design decisions

Decision	Rationale
Crypto verification in Secure world, before NS boots	Strongest security boundary — NS never runs untrusted code
NS is write-only	Simpler NS firmware; no rollback logic; no EEPROM flags
SWAP_BANK as implicit FOTA-pending signal	No separate flag needed; SWAP_BANK=1 means "verify before booting"
No boot attempt counter	Crypto integrity is the only gate; runtime health is the application's responsibility
wolfSSL with WOLFSSL_STATIC_MEMORY	Project requirement: no malloc/free anywhere

2. Memory Layout

Flash banks (STM32H563 — 2 MB device)

```
Bank 1 (1 MB physical — 0x08000000):
  Secure alias: 0x0C000000 (ALWAYS Bank 1, regardless of SWAP_BANK)
  └─ Sectors 0-30 (248KB) — Secure firmware code + wolfSSL
  └─ Sector 31 (8KB) — NSC veneer (SECWM1_END = 0x1F)
  └─ Sectors 32-119 (704KB) — NS firmware (active when SWAP_BANK=0)
    NSBOOTADD = 0x80400 → 0x08040000
  └─ Sectors 120-127 (EDATA, 48KB) — Calibration (stable, never swaps)

Bank 2 (1 MB physical — 0x08100000):
  Fully Non-Secure (SECWM2_STRT=0x7F, SECWM2_END=0x00)
  └─ Sectors 0-31 (256KB) — Reserved (mirrors Secure offset, not used)
  └─ Sectors 32-119 (704KB) — FOTA target: new NS firmware
    Written by SECURE_FOTA_WriteChunk()
    Active when SWAP_BANK=1
```

Address map

Address	SWAP_BANK=0	SWAP_BANK=1	Purpose
0x0C000000	Bank 1 physical	Bank 1 physical	Secure boot — always Bank 1
0x08040000	Bank 1 sector 32 (active NS)	Bank 2 sector 32 (new NS)	NS firmware entry
0x08140000	Bank 2 sector 32 (FOTA target)	Bank 1 sector 32 (old NS)	FOTA write target
0x0D000000	Bank 1 EDATA (Secure alias)	Bank 1 EDATA (Secure alias)	Calibration — never changes

Critical: 0x0C000000 (Secure alias) always maps to Bank 1 physical, regardless of SWAP_BANK. This means the Secure firmware is safe to use SWAP_BANK for NS firmware updates — it always boots from Bank 1.

Option bytes

Option byte	Value	Meaning
SECWM1_END	0x1F	Secure watermark covers sectors 0-31 (256KB)
NSBOOTADD	0x80400	NS boot address = 0x08040000 (sector 32)
SECB00TADD	0x0C0000	Secure boot address = 0x0C000000
SWAP_BANK	0x0 (normal) / 0x1 (FOTA staged)	Bank swap state

Linker scripts

Secure/STM32H563xx_FLASH.s.ld:

```
FLASH      (rx)   : ORIGIN = 0x0C000000, LENGTH = 248K
FLASH_NSC  (rx)   : ORIGIN = 0x0C03E000, LENGTH = 8K
EDATA_S    (rw)   : ORIGIN = 0x0D000000, LENGTH = 48K
```

NonSecure/STM32H563xx_FLASH_ns.s.ld:

```
FLASH  (rx)   : ORIGIN = 0x08040000, LENGTH = 704K
```

3. Boot Sequence

Normal boot (SWAP_BANK=0)

```
Power on
→ Secure firmware starts at 0x0C000000 (Bank 1)
→ fota_boot_check(): SWAP_BANK=0 → no FOTA pending → skip verification
→ Configure SAU, GTZC, GPIO, EEPROM emulation
→ Boot NS firmware at 0x08040000 (Bank 1 sector 32)
→ NS firmware runs normally
```

FOTA download (after SECURE_FOTA_Stage())

```
NS firmware receives firmware via HTTP POST
→ SECURE_FOTA_EraseTarget(): erase inactive bank
→ SECURE_FOTA_WriteChunk(): write new firmware to inactive bank
→ SECURE_FOTA_Stage(): write FOTA_PENDING=0xF0F0F0F0 to EEPROM, return
```

```
FOTA_OK
→ HTTP 200 "FOTA downloaded, activates on next power cycle"
→ Current firmware KEEPS RUNNING – no reset, no bank swap
→ New firmware activates only on next external POR
```

External POR boot (FOTA_PENDING=0xF0F0F0F0)

```
External power cycle (POR)
→ Secure firmware starts at 0x0C000000 (Bank 1 – always)
→ fota_boot_check(): read FOTA_PENDING from EEPROM
    FOTA_PENDING == 0xF0F0F0F0 → FOTA pending → activate + verify
    Toggle SWAP_BANK (HAL_FLASHEx_OBProgram + HAL_FLASH_OB_Launch)
    Read firmware package from 0x08040000 (now points to new bank)
    Parse trailer: cert_size + magic "FOTA"
    Verify X.509 cert against embedded CA cert (wolfSSL)
    Compute SHA-256 of firmware, compare with hash in cert
    └ VALID: clear FOTA_PENDING=0 → boot NS from new firmware
    └ INVALID: toggle SWAP_BANK back + clear FOTA_PENDING=0
                → boot NS from old firmware (no additional reset needed)
→ Configure SAU, GTZC, GPIO, EEPROM emulation
→ Boot NS firmware at 0x08040000
→ NS firmware runs
```

Normal boot (FOTA_PENDING=0)

```
Power on (no FOTA pending)
→ Secure firmware starts at 0x0C000000
→ fota_boot_check(): FOTA_PENDING != 0xF0F0F0F0 → skip
→ Boot NS normally
```

Why FOTA_PENDING is needed (not just SWAP_BANK)

SWAP_BANK alternates: 0 → 1 → 0 → 1 with each FOTA. Using SWAP_BANK=1 as the verification trigger would miss every other FOTA (v2 → v3 goes 1 → 0).

FOTA_PENDING is an explicit "activate + verify on next POR" flag that is:

- **Set** by `SECURE_FOTA_Stage()` on every FOTA download
- **Cleared** by Secure `main.c` after activation (pass or fail)
- **Independent** of SWAP_BANK value
- **Survives** software resets (EEPROM is persistent)

State trace for 3 sequential FOTAs:

Step	Action	SWAP_BANK	FOTA_PENDING	Active firmware
1	Flash v1 (virgin MCU)	0	0	Bank 1 → v1

Step	Action	SWAP_BANK	FOTA_PENDING	Active firmware
2	Download v2 (FOTA_Stage)	0	0xF0F0F0F0	Bank 1 → v1 (still running)
3	External POR: toggle+verify v2 → valid	1	0	Bank 2 → v2
4	Download v3 (FOTA_Stage)	1	0xF0F0F0F0	Bank 2 → v2 (still running)
5	External POR: toggle+verify v3 → valid	0	0	Bank 1 → v3
6	Download v4 (FOTA_Stage)	0	0xF0F0F0F0	Bank 1 → v3 (still running)
7	External POR: toggle+verify v4 → invalid	0	0	Bank 1 → v3 (rollback)

4. Firmware Package Format

Offset	Content
0	Raw NS firmware binary (padded to 16-byte boundary with 0xFF)
fw_size	X.509 certificate DER (contains SHA-256 hash in custom extension)
total - 8	cert_size (uint32_t, little-endian)
total - 4	magic (uint32_t = 0x464F5441 = "FOTA")

Custom X.509 extension:

- OID: 1.3.6.1.4.1.99999.1 (private enterprise, Jerry FOTA)
- DER encoding: 06 0A 2B 06 01 04 01 86 8D 1F 01 01
- Value: OCTET STRING { SHA-256(firmware_binary) }

Verification steps in fota_boot_check() (Secure main.c):

1. Read SWAP_BANK option byte — if 0, skip (no FOTA pending)
2. Read 8-byte trailer from 0x08040000 + total_size - 8
3. Validate magic "FOTA" and cert_size
4. Parse X.509 cert with wolfSSL DecodedCert
5. Verify cert signature against embedded CA cert (fota_ca_cert_der[])
6. Extract SHA-256 hash from custom OID extension
7. Compute SHA-256 of firmware bytes in flash
8. Compare computed hash vs hash in cert
9. Valid → continue boot
10. Invalid → toggle SWAP_BANK=0 + reset

5. NSC Gateway Interface

Declared in `secure_nsc.h`, implemented in `secure_nsc.c`.

```
/* Erase inactive bank sectors 32-119 (704KB NS firmware area) */
uint32_t SECURE_FOTA_EraseTarget(void);

/* Write a chunk of firmware data to the inactive bank */
uint32_t SECURE_FOTA_WriteChunk(uint32_t offset, const uint8_t *pData,
uint32_t len);

/* Stage the firmware: write FOTA_PENDING=0xF0F0F0F0 to EEPROM and return.
 * Returns FOTA_OK on success, FOTA_ERR_* on failure.
 *
 * Does NOT toggle SWAP_BANK. Does NOT reset.
 * Current firmware keeps running after this call.
 *
 * On the next external POR, Secure main.c reads FOTA_PENDING, toggles
SWAP_BANK,
 * verifies the staged firmware, and boots NS from the new bank (or rolls
back).
 *
 * No crypto verification here – verification happens in Secure main.c on
next POR. */
uint32_t SECURE_FOTA_Stage(void);
```

Removed from v1:

- `SECURE_FOTA_Commit()` — replaced by `SECURE_FOTA_Stage()` (no crypto)
- `SECURE_FOTA_Rollback()` — rollback now handled by Secure `main.c`
- `SECURE_CAL_GetSwapBank()` — no longer needed

EEPROM flag:

- `FOTA_PENDING_VADDR = 0x0030` — replaces `FOTA_VALID_FLAG_VADDR`
- Value `0xF0F0F0F0` = FOTA staged, verify on next boot
- Value `0x00000000` (or any other) = no FOTA pending
- Set by `SECURE_FOTA_Stage()`, cleared by Secure `main.c` after verification

FOTA target address (dynamic)

```
// Always writes to the INACTIVE bank's NS region
static uint32_t fota_get_target_base(void)
{
    bool swapped = (READ_BIT(FLASH->OPTSR_CUR, FLASH_OPTSR_SWAP_BANK) !=
0U);
    uint32_t bank_base = swapped ? 0x08000000UL : 0x08100000UL;
    return bank_base + (32U * 8192U); // skip sectors 0-31 (Secure
```

```
partition)
}
```

6. NS FOTA Task

application/src/fota_task.c

```
void vFotaTask(void *pvParameters)
{
    // Wait for all tasks to sync
    xEventGroupSync(...);

    // Start the FOTA HTTP server – blocks indefinitely
    // No startup check, no rollback, no EEPROM flags
    fota_http_server_run();
}
```

NS firmware responsibilities:

- Receive firmware via HTTP POST
- Write chunks to inactive bank via `SECURE_FOTA_WriteChunk()`
- Call `SECURE_FOTA_Stage()` to trigger bank swap + reset
- **Nothing else** — no crypto, no rollback, no health checks

7. HTTP Server Protocol

Endpoint: `POST /fota` on port `8080`

Request:

```
POST /fota HTTP/1.1
Content-Type: application/octet-stream
Content-Length: <size>
Expect: 100-continue

<binary body: signed firmware package>
```

Server state machine:

```
RECV headers → validate POST /fota + Content-Length
    → send 100 Continue (if Expect header present)
    → SECURE_FOTA_EraseTarget()
    → stream body to flash (4KB chunks via SECURE_FOTA_WriteChunk)
    → flush final partial chunk (pad to 16 bytes with 0xFF)
    → SECURE_FOTA_Stage() ← write FOTA_PENDING=0xF0F0F0F0, returns FOTA_OK
```

```

or FOTA_ERR_*
→ if FOTA_OK:
    send HTTP 200 "FOTA downloaded, activates on next power cycle"
    current firmware KEEPS RUNNING – no reset
→ if FOTA_ERR_*:
    send HTTP 400 "FOTA staging failed: <error code>"
    (FOTA_PENDING not set – device stays on current firmware)

```

Note: The new firmware activates only on the next **external POR (power cycle)**. Software resets (watchdog, NVIC_SystemReset) do NOT activate the new firmware. This ensures the current firmware remains operational until a deliberate power cycle.

Responses:

Status	Meaning
100 Continue	Sent when Expect: 100-continue is present
200 OK	Firmware staged, device rebooting
400 Bad Request	(future: staging error)
404 Not Found	POST to wrong path
405 Method Not Allowed	Non-POST request
411 Length Required	Missing Content-Length
500 Internal Server Error	Flash erase or write failed

8. Cryptographic Design

Algorithm choices

Component	Algorithm	Reason
Signature	ECDSA-P256	Small key size, fast verify, widely supported
Hash	SHA-256	Standard, hardware-acceleratable on STM32H5
Certificate	X.509 DER	Standard format; openssl tooling available

wolfSSL configuration

File: **user_settings.h**

Key settings:

- **WOLFSSL_STATIC_MEMORY** + **WOLFSSL_NO_MALLOC** — fully static, 8KB pool in **.bss**
- **WOLFCRYPT_ONLY** — no TLS stack, no **sys/socket.h**
- **HAVE_ECC** + **HAVE_ECC_VERIFY** — ECDSA-P256 verify only
- **WOLFSSL_SHA256** — SHA-256
- **WC_NO_RNG** — no RNG needed for verify-only

- `NO_ASN_TIME` — no wall clock, skip cert validity period check
- `WOLFSSL_SINGLE_THREADED` — no pthreads

Certificate chain



Hardware acceleration upgrade path

To enable STM32H5 HASH/PKA hardware acceleration, add to `user_settings.h`:

```
#define WOLFSSL_STM32H5
#define WOLFSSL_STM32_HASH // SHA-256 via STM32H5 HASH peripheral
#define WOLFSSL_STM32_PKA // ECDSA via STM32H5 PKA peripheral
```

No changes to `fota_crypto.c` or `secure_nsc.c` required.

9. File Map

Secure firmware files

File	Purpose
<code>Secure/Core/Src/main.c</code>	Boot sequence — <code>fota_boot_check()</code> before NS boot
<code>Secure/Core/Src/secure_nsc.c</code>	NSC gateway: EraseTarget, WriteChunk, Stage
<code>Secure/Core/Src/fota_crypto.c</code>	wolfSSL wrappers: SHA-256, ECDSA-P256, X.509
<code>Secure/Core/Inc/fota_crypto.h</code>	Crypto API header
<code>Secure/Core/Inc/user_settings.h</code>	wolfSSL configuration
<code>Secure_nsclib/secure_nsc.h</code>	NSC declarations (shared with NS)

NS firmware files

File	Purpose
------	---------

File	Purpose
application/src/fota_task.c	FreeRTOS task — starts HTTP server
application/src/fota_http_server.c	lwIP HTTP server — receive + write + stage
application/inc/fota_http_server.h	HTTP server header

Tools

File	Purpose
tools/sign_firmware.py	Sign firmware binary → signed package
tools/fota_upload.sh	Sign + upload in one command
keys/fota_ca.crt	CA certificate (public, committed)
keys/fota_ca.key	CA private key (gitignored)

10. End-to-End Workflow

One-time setup

```
# Generate CA key pair
openssl ecparam -name prime256v1 -genkey -noout -out keys/fota_ca.key
openssl req -new -x509 -key keys/fota_ca.key -out keys/fota_ca.crt \
    -subj "/CN=Jerry FOTA CA" -days 3650

# Export DER bytes for embedding in secure_nsc.c
openssl x509 -in keys/fota_ca.crt -outform DER | xxd -i
# → paste into fota_ca_cert_der[] in secure_nsc.c
```

Per-release update

```
# Build firmware
uv run python tools/build.py build

# Sign + upload (one command)
./tools/fota_upload.sh <device-ip>

# Or manually:
uv run python tools/sign_firmware.py \
    --input build/stm-Debug/application/jerry_app.bin \
    --ca-key keys/fota_ca.key \
    --ca-cert keys/fota_ca.crt \
    --output jerry_app_signed.bin

curl -X POST http://<device-ip>:8080/fota \
```

```
-H "Content-Type: application/octet-stream" \  
--data-binary @jerry_app_signed.bin
```

What happens after curl sends the firmware

- 1. NS receives firmware via HTTP POST
- 2. NS writes chunks to inactive bank (SECURE_FOTA_WriteChunk)
- 3. NS sends HTTP 200 "FOTA staged, rebooting"
- 4. NS calls SECURE_FOTA_Stage() → SWAP_BANK toggled → RESET (immediate)
- 5. Secure firmware boots (always from Bank 1 @ 0x0C000000)
- 6. fota_boot_check(): SWAP_BANK=1 → verify staged firmware
 - a. Parse trailer, verify X.509 cert, compare SHA-256
 - b. VALID: boot NS from new firmware (Bank 2 sector 32)
 - c. INVALID: toggle SWAP_BANK=0 + reset → old firmware boots
- 7. NS firmware runs

11. Error Codes

Code	Value	Meaning
FOTA_OK	0	Success
FOTA_ERR_FLASH_UNLOCK	1	HAL_FLASH_Unlock() failed
FOTA_ERR_ERASE	2	Sector erase failed
FOTA_ERR_WRITE	3	Flash program failed
FOTA_ERR_BAD_POINTER	4	pData not in NonSecure memory
FOTA_ERR_ALIGNMENT	5	offset or len not 16-byte aligned
FOTA_ERR_BAD_SIZE	6	total_size too small or cert_size invalid
FOTA_ERR_BAD_MAGIC	7	Trailer magic ≠ "FOTA"
FOTA_ERR_CERT_PARSE	8	wolfSSL failed to parse firmware cert
FOTA_ERR_CA_PARSE	9	wolfSSL failed to parse CA cert
FOTA_ERR_CERT_VERIFY	10	Firmware cert signature invalid
FOTA_ERR_NO_HASH	11	SHA-256 hash extension not found in cert
FOTA_ERR_HASH_COMPUTE	12	SHA-256 computation error
FOTA_ERR_HASH_MISMATCH	13	Computed hash ≠ hash in cert

12. Changes from v1

v1	v2
----	----

v1	v2
<code>SECURE_FOTA_Commit()</code> — verify + swap + reset	<code>SECURE_FOTA_Stage()</code> — swap + reset only (no crypto)
<code>SECURE_FOTA_Rollback()</code> — NS-callable rollback	Removed — rollback handled by Secure <code>main.c</code>
<code>SECURE_CAL_GetSwapBank()</code> — NS reads SWAP_BANK	Removed — not needed
<code>FOTA_VALID_FLAG_VADDR</code> — "firmware is valid" flag	<code>FOTA_PENDING_VADDR</code> — "FOTA staged, verify on next boot" flag
<code>fota_startup_check()</code> — NS checks flag on boot	Removed — Secure <code>main.c</code> checks FOTA_PENDING before NS boots
<code>fota_mark_valid()</code> — NS marks firmware valid	Removed — Secure <code>main.c</code> clears FOTA_PENDING after verification
Crypto verification in NS (via NSC)	Crypto verification in Secure <code>main.c</code> before NS boots
NS has rollback logic	NS is write-only — no rollback, no checks
SWAP_BANK=1 as verification trigger	FOTA_PENDING flag as verification trigger (works for all SWAP_BANK values)